

Институт перспективных технологий
и индустриального программирования

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт перспективных технологий и индустриального программирования

Кафедра индустриального программирования



**«Разработка веб-приложения для управления
техническим обслуживанием и ремонтом
оборудования предприятия»**

Подготовил:

студент группы ЭФБО-06-22

Суднишников Никита Михайлович

Руководитель:

к.т.н., доцент кафедры индустриального программирования

Бочаров Михаил Иванович

ВМЕСТЕ СОЗДАЁМ БУДУЩЕЕ

Актуальность исследования

На промышленных предприятиях на техническое обслуживание и ремонт оборудования уходит **от 15 до 40 процентов** операционного бюджета. Бумажный документооборот замедляет обработку заявок в несколько раз и приводит к потере **до 20 процентов обращений**. Готовые CMMS-системы либо стоят **миллионы рублей**, либо хранят данные **за рубежом**. Для заводов среднего масштаба (200–800 единиц оборудования) доступной отечественной альтернативы нет.

Сокращение времени обработки заявки **с часов до минут**

Централизованный учёт 200–300 единиц оборудования

Автоматическое планирование регламентного ТО

Импортонезависимость — данные на своих серверах

Объект, предмет, цель и задачи

Объект исследования:

процессы технического обслуживания и ремонта оборудования предприятия

Предмет исследования:

веб-приложение для управления процессами ТОиР

Цель работы:

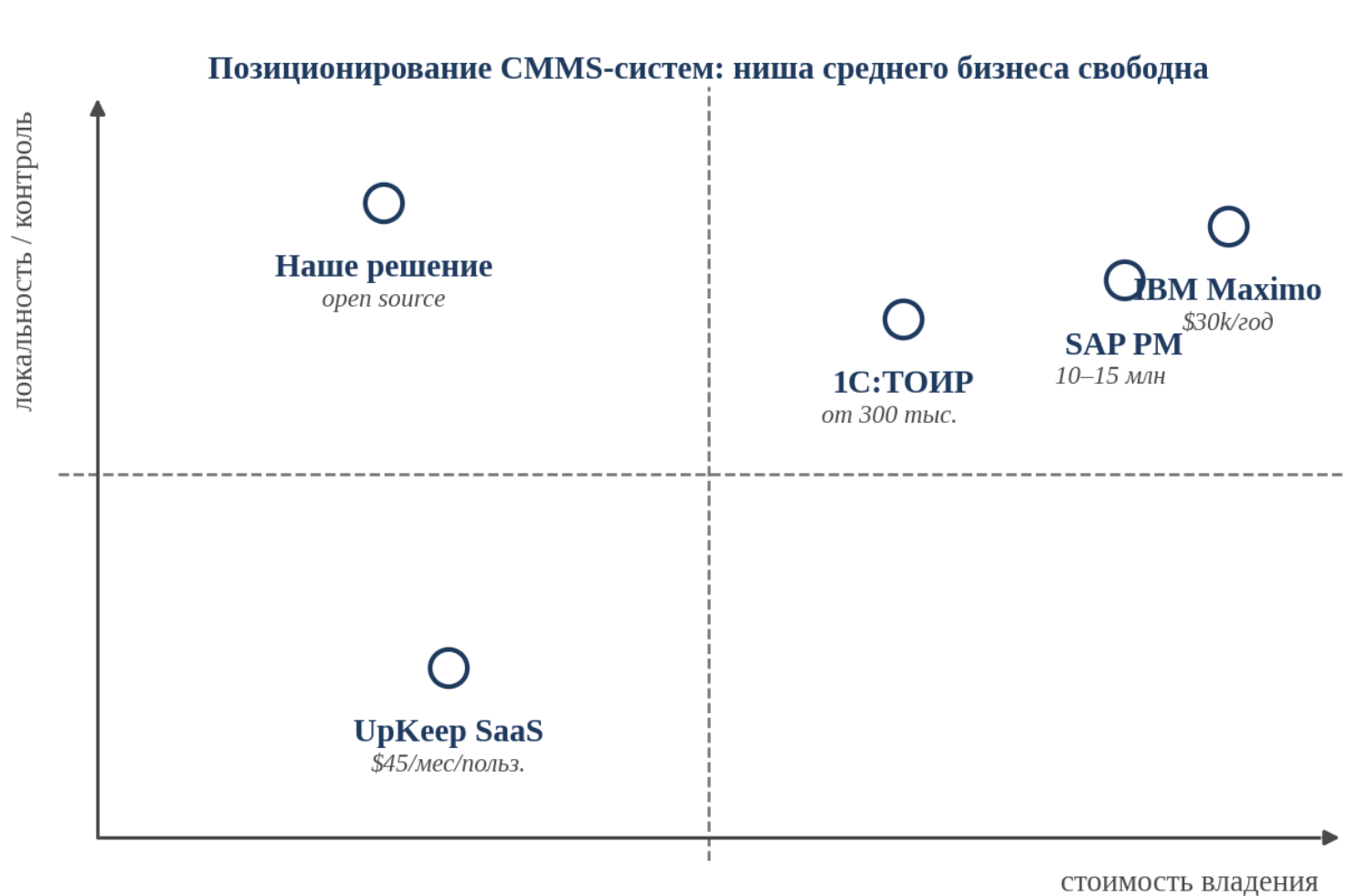
повышение эффективности процессов технического обслуживания и ремонта оборудования предприятия за счёт разработки и внедрения веб-приложения, обеспечивающего **централизованный учёт** оборудования и его технического состояния, **планирование и контроль** выполнения регламентных и внеплановых работ, **автоматизацию формирования заявок** на ремонт, а также повышение прозрачности и управляемости взаимодействия между техническими службами и эксплуатирующими подразделениями.

Задачи работы:

1. Провести анализ предметной области ТОиР и выявить ключевые проблемы учёта, планирования и координации ремонтных работ.
2. Исследовать существующие цифровые решения и подходы (CMMS/EAM), оценить их достоинства и ограничения.
3. Сформировать функциональные и нефункциональные требования.
4. Разработать архитектуру программного решения и структуру основных компонентов.
5. Спроектировать базу данных для учёта оборудования, регламентных работ, заявок и истории обслуживания.
6. Реализовать веб-приложение для учёта, планирования, обработки заявок и формирования отчётности.
7. Провести тестирование разработанного решения и оценить его практическую применимость.

Обзор CMMS-систем

Пустая ниша для среднего бизнеса



1C:ТОИР — от 300 тыс. ₽, требует штатного 1C-программиста и сервера приложений.
SAP PM — 10–15 млн ₽ только за лицензию, внедрение от полугода.
IBM Maximo — 30 тыс. \$/год, интеграции под крупные холдинги.
UpKeep (SaaS) — 45 \$/мес за пользователя, данные хранятся за рубежом.
Для завода на **200–800 станков** и **5–15 ремонтников** доступного отечественного on-premise-решения на рынке нет.

Вывод: собственная разработка закрывает нишу, где крупные CMMS избыточны, а облачные сервисы неприменимы.

Технологический стек

Пять слоёв: от SPA до контейнеризации

Уровни системы: фронт → прокси → API → БД → контейнеризация

Vue 3 + Pinia

Composition API · 98 компонентов · бандл 142 КБ gzip



Nginx

reverse проху · отдача SPA · проксирование /api · SSL



Go 1.22 + Echo

Clean Architecture · 32 REST-эндпоинта · JWT



PostgreSQL 16

ACID · JSONB · 8 сущностей · 12 таблиц



Docker Compose

multi-stage · образ Go 18 МБ · развёртывание в 1 команду

Go + Echo — один бинарник 18 МБ, горутины держат 500+ соединений при медиане 47 мс.
Vue 3 + Pinia (Composition API) — ядро 33 КБ, итоговый бандл 142 КБ gzip.

PostgreSQL 16 — ACID-транзакции, JSONB для расширяемых полей оборудования.

Nginx — reverse проху: раздача SPA и проксирование /api на бэкэнд.

Docker Compose — multi-stage сборка, развёртывание одной командой.

Отвергнуты: Python (GIL тормозит конкурентность), Node.js (single-thread под нагрузкой), React (на 40 % тяжелее Vue).

Роли пользователей и требования

4 роли, 8 ключевых сценариев, RBAC на 16 ключах



Оператор — создаёт заявки на ремонт, отслеживает их статус.

Техник — принимает заявки в работу, фиксирует результаты.

Инженер — планирует регламентное ТО, смотрит отчётность и дашборд.

Администратор — управляет пользователями, ролями, справочниками.

Всего: **15 функциональных** и **12 нефункциональных** требований.

Авторизация — JWT с refresh-ротацией. Доступ — модель RBAC на 16 ключах, подсистема привязана к ролям.

Пользовательские истории

User Story Map: 5 этапов × 3 уровня приоритета

User Story Map: 5 этапов × 3 уровня приоритета = 15 историй

этап	Аутентификация	Оборудование	Заявки	Планирование ТО	Аналитика
MVP	Вход по логину и паролю	Просмотр списка	Создание заявки	График ТО	Дашборд KPI
расширение	Сброс пароля	Редактирование карточки	Назначение исполнителя	Автоматические наряды	Отчёт за период
развитие	Управление ролями	История обслуживания	Уведомления о сроках	Напоминания по ППР	Экспорт в Excel

Backbone (этапы пути): аутентификация → оборудование → заявки → планирование ТО → аналитика.

MVP — минимальный функционал каждого этапа (5 историй).

Расширение — второй приоритет (назначение, автонаряды, отчёты).

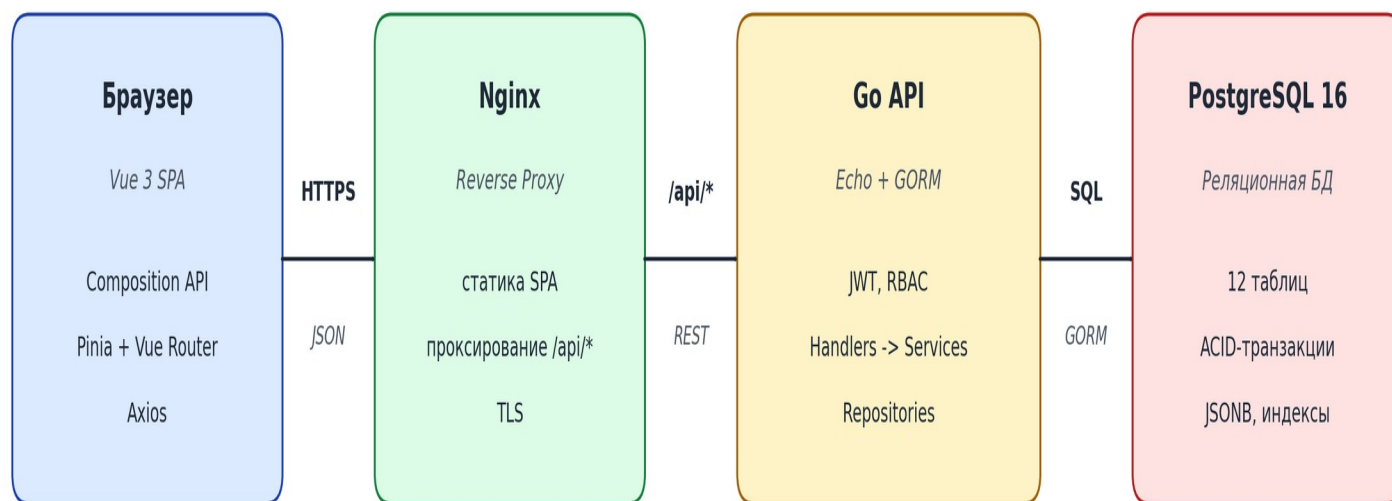
Развитие — уведомления, напоминания, экспорт, аудит.

Карта связывает требования, этапы разработки и сценарии тестирования.

Каждая история прописана в формате «Как %роль%, я хочу %действие%, чтобы %результат%» и покрыта тестами.

Архитектура приложения

Clean Architecture на Go, Vue 3 SPA через Nginx



Контейнеризация: Docker Compose (api, frontend, db)

Слой бэкенда: handler → service → repository с инверсией зависимостей.

62 Go-файла в 14 пакетах обслуживают **32 REST-эндпоинта**.

98 Vue-компонентов, Pinia-сторы, ленивая загрузка админ-модулей.

Nginx работает обратным прокси: SPA single-origin, проксирование /api.

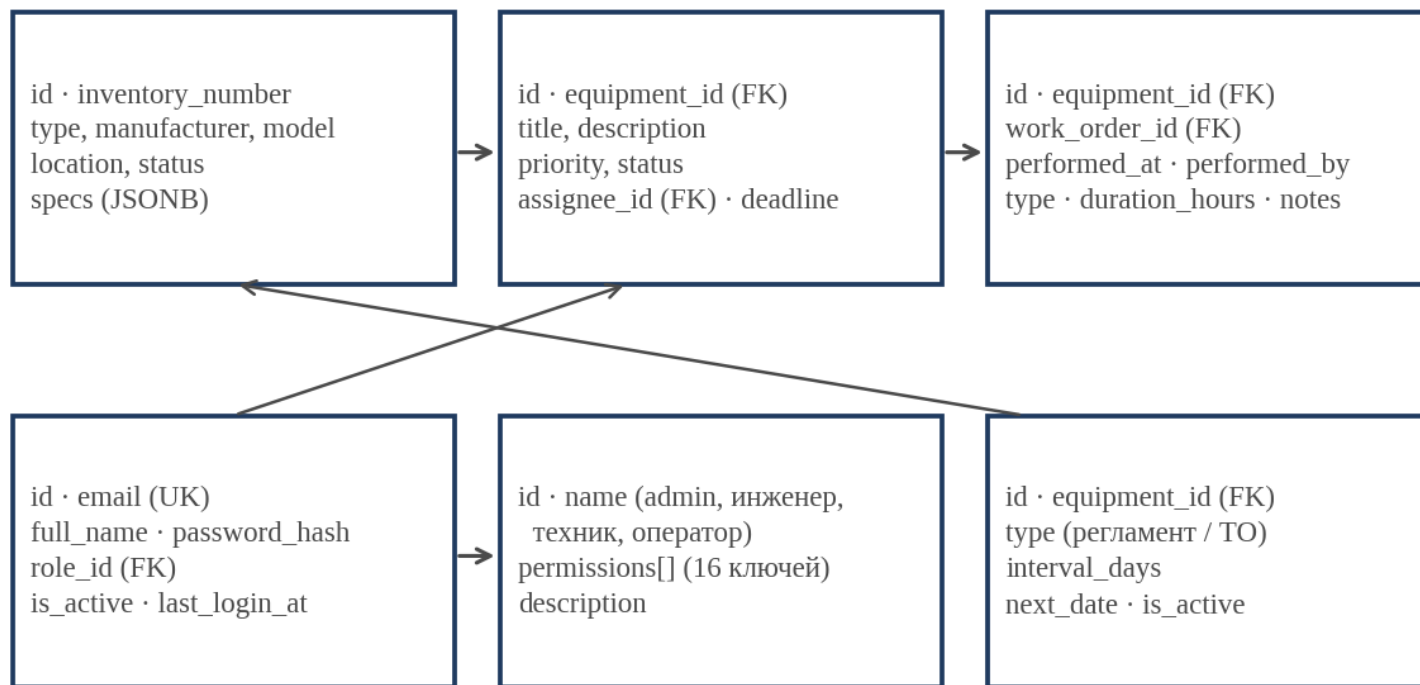
Инфраструктура устойчивости: **rate limit 120 req/min**, graceful shutdown, структурированные логи (zap).

Все 32 эндпоинта задокументированы в Swagger; из того же OpenAPI-спека можно генерировать типобезопасных клиентов.

Модель данных

6 центральных сущностей, 8 всего, 12 таблиц

Модель данных: 6 центральных сущностей, ещё 2 таблицы-связки



всего 8 сущностей · 12 таблиц · foreign keys с каскадным поведением

equipment — паспорт станка, JSONB-поле для расширяемых характеристик.

work_order — заявка на ремонт с FK на оборудование и исполнителя.

maintenance_history — журнал фактических работ и их длительности.

maintenance_plan — регламент ТО: интервал, следующая дата, активность.

user, role — авторизация, 4 роли и 16 разрешений.

Foreign keys с каскадным поведением, индексы на частые запросы (по статусу, исполнителю, сроку).

Интерфейс: работа с заявками

Форма создания, список с фильтрами, 6 состояний наряда

The screenshot shows a web interface for creating a new repair request. On the left is a dark sidebar with the 'ТОиР' (TOiP) logo and user role 'Engineer'. The sidebar menu includes 'Дашборд' (Dashboard), 'Оборудование' (Equipment), 'Заявки на ремонт' (Repair requests), and 'Графики ТО' (Maintenance graphs). The main content area is titled 'Новая заявка на ремонт' and includes a link to the request list. The form contains the following fields: 'Оборудование *' (Equipment) with a dropdown menu, 'Заголовок *' (Header) with a text input for a brief problem description, 'Описание' (Description) with a text area for a detailed problem description, and 'Приоритет' (Priority) with a dropdown menu currently set to 'Средний (medium)'. At the bottom right of the form are 'Отмена' (Cancel) and 'Создать заявку' (Create request) buttons.

Форма заявки — выбор оборудования из справочника, тип проблемы, срочность, фото.
Список заявок — фильтры по статусу, исполнителю, типу оборудования; подсветка просроченных.

Состояния: новая → принята → в работе → на проверке → закрыта → архив.

Уведомления исполнителю при назначении и за сутки до дедлайна.

Kanban-режим для диспетчера (опциональный).

Валидация на клиенте и сервере. Срок выполнения рассчитывается из приоритета и регламента предприятия.

Интерфейс: оборудование и ТО

Карточки, списки, автоматические графики ППР

ТОиР

Engineer

■ Дашборд

● Оборудование

📅 Заявки на ремонт

📅 Графики ТО

Выход

← К списку оборудования

active Редактировать

Токарный станок 16K20

ИНВЕНТАРНЫЙ НОМЕР	ТС-001	НАИМЕНОВАНИЕ	Токарный станок 16K20
ТИП	---	МОДЕЛЬ	16K20
ПРОИЗВОДИТЕЛЬ	СВЭС	СЕРИЙНЫЙ НОМЕР	SN-TC-001
МЕСТОПОЛОЖЕНИЕ	Цех 1	ДАТА УСТАНОВКИ	2024-04-12T18:41:39.706254Z
ПОСЛЕДНЕЕ ТО	2026-03-12T18:41:39.706254Z	ПРИМЕЧАНИЯ	---

История обслуживания

История обслуживания будет отображена здесь

Карточка оборудования — паспорт, журнал работ, расписание ТО, вложения.

Список — сортировка по цеху, состоянию, наработке.

Графики ТО — расчёт дат по циклу и наработке; автоматические наряды.

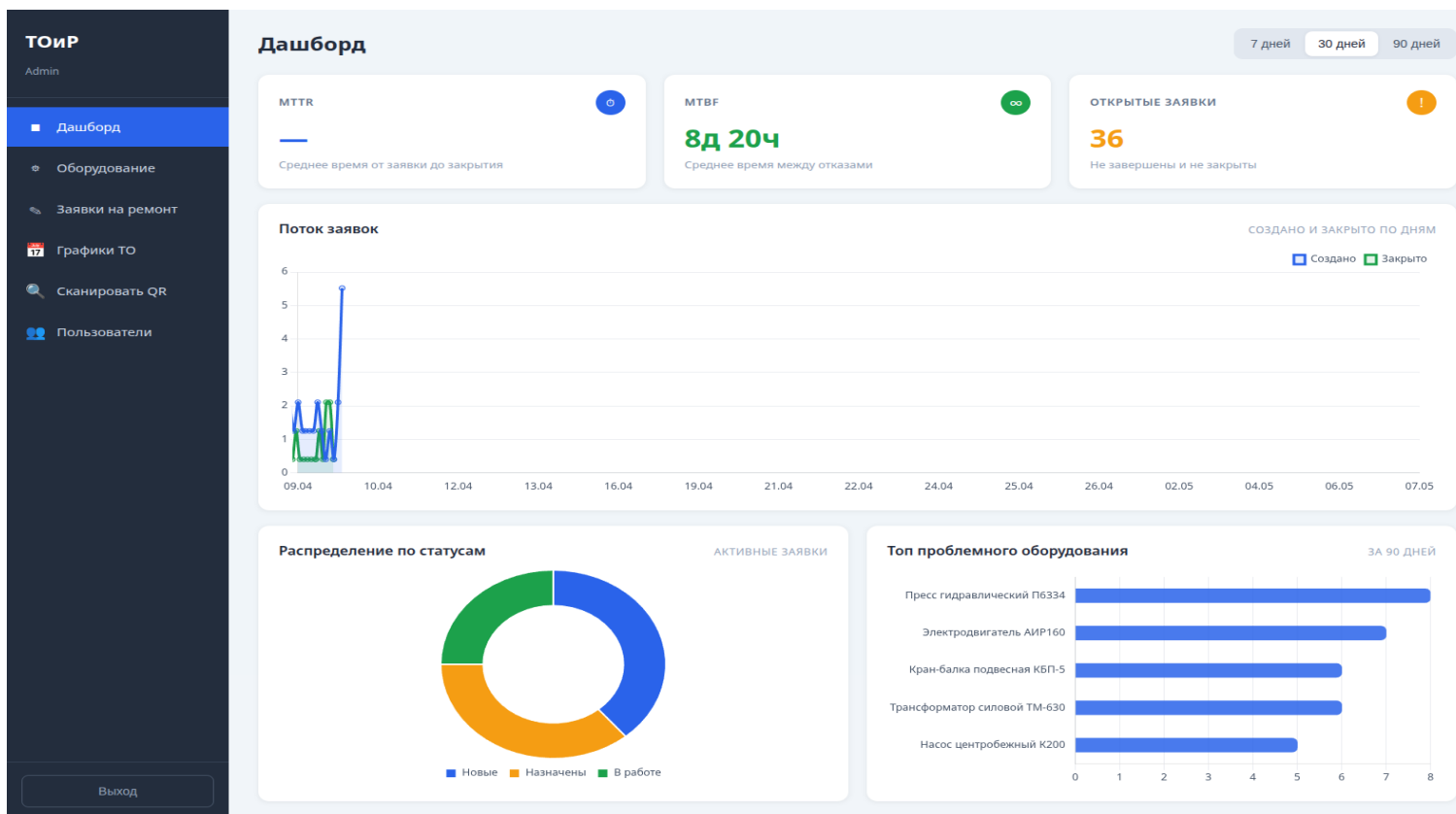
История обслуживания доступна одним кликом из карточки.

Регламент настраивается под конкретную модель станка.

Система напоминает о предстоящем обслуживании и формирует наряды по расписанию без участия инженера.

Дашборд и REST API

KPI в реальном времени, 32 эндпоинта в Swagger



Дашборд — активные заявки, загрузка техников, процент выполнения ППР. **Период и фильтры** настраиваемы, данные обновляются в реальном времени. **32 REST-эндпоинта** задокументированы в Swagger UI (OpenAPI 3). **JWT** хранится в httpOnly-cookie, активна CSRF-защита и строгие CSP-заголовки. API доступен сторонним системам (1С, IoT-шлюзы) через отдельный API-ключ.

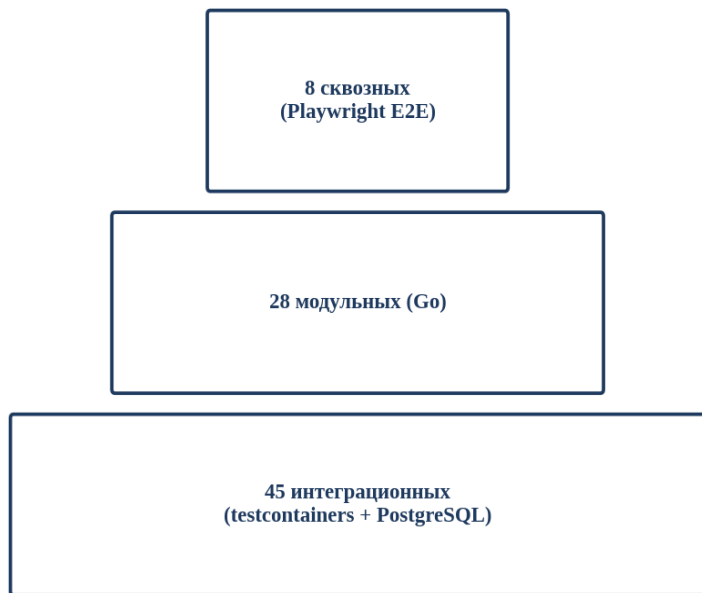
Swagger UI доступен только администратору и инженеру. Из OpenAPI-спека генерируется типобезопасный TS-клиент.

Тестирование и нагрузка

81 тест, покрытие 68 %, медиана 47 мс при 500 подключениях

Пирамида тестов

81 тест · 100 % пройдено



Покрывтие кода



Нагрузка · 500 одновременных подключений



28 модульных тестов (Go) — доменная логика и сервисы.

45 интеграционных (testcontainers + PostgreSQL) — репозитории и API.

8 сквозных (Playwright) — ключевые пользовательские сценарии.

Покрывтие критичных модулей 76–84 % (аутентификация, обработка заявок).

Нагрузка: 500 одновременных подключений, p95 = 180 мс, throughput 2.4k req/s.

Все 81 тест проходят в CI. Docker-образы: Go API 18 МБ, Nginx+Vue 25 МБ. Развёртывание — одна команда.

«Разработка веб-приложения для управления техническим обслуживанием
и ремонтом оборудования предприятия»

Спасибо за внимание

Подготовил:

студент группы ЭФБО-06-22
Суднишников Никита Михайлович

Руководитель:

к.т.н., доцент кафедры индустриального программирования
Бочаров Михаил Иванович

ВМЕСТЕ СОЗДАЁМ БУДУЩЕЕ